Learning Quantum Dynamics via Fourier Coefficient Extraction

Theory, algorithms and implementation

After A. Barthe, M. Yaghubi Rad, M. Grossi & V. Dunjko, `arXiv:2506.17089` (2025)

Thomas Lagoutte

M2 *Quantum Devices, Materials and Information* (QMI) — Institut Polytechnique de Paris

Internship at ICEPP, The University of Tokyo

Supervisor: **Prof. Koji Terashi**

April 30, 2026

Roadmap. **I.** Theory & proofs (PAC, Algorithm A, classical hardness) • **II.** Implementation stack (Qiskit, registers, pipeline) • **III.** Numerical results (TFIM, Schwinger, shot-noise)

INSTITUT POLYTECHNIQUE DE PARIS

東京大学 THE UNIVERSITY OF TOKYO

東京大学 素粒子物理国際研究センター International Center for Elementary Particle Physics The University of Tokyo

Motivation — Learning Quantum Dynamics from Classical Data

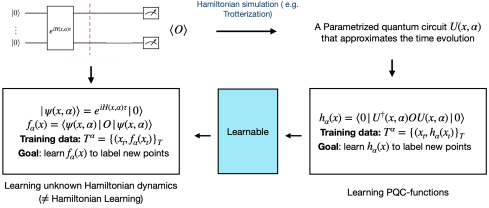
Supervised task. Given i.i.d. samples $\mathcal{D}_T = \{(\boldsymbol{x}_t, y_t)\}_{t=1}^{T} \sim \mathcal{D}^T$ with labels $y_t = f_{\boldsymbol{\alpha}}(\boldsymbol{x}_t)$, where

$$f_{\boldsymbol{\alpha}}(\boldsymbol{x}) \;=\; \langle\psi(\boldsymbol{x},\boldsymbol{\alpha})|\; O\;|\psi(\boldsymbol{x},\boldsymbol{\alpha})\rangle\,, \qquad |\psi(\boldsymbol{x},\boldsymbol{\alpha})\rangle = e^{\,i\,H(\boldsymbol{x},\boldsymbol{\alpha})\,t}\,|\psi_0\rangle\,,$$

return h that ε-approximates $f_{\boldsymbol{\alpha}}$ on unseen $\boldsymbol{x} \sim \mathcal{D}$ (PAC sense, formalised next slide).

Two faces of the same concept class

- Hamiltonian dynamics (physical): $f_{\boldsymbol{\alpha}}(\boldsymbol{x}) = \langle\psi(\boldsymbol{x},\boldsymbol{\alpha})|\, O\,|\psi(\boldsymbol{x},\boldsymbol{\alpha})\rangle$.
- PQC functions (algorithmic): $h_{\boldsymbol{\alpha}}(\boldsymbol{x}) = \langle 0|\, U^\dagger(\boldsymbol{x},\boldsymbol{\alpha})\, O\, U(\boldsymbol{x},\boldsymbol{\alpha})\,|0\rangle$.
- Bridged by Hamiltonian simulation (Trotter, qubitization, LCU).



Bridge between physical dynamics and PQC functions; PAC-learnability transfers across it. Adapted from Barthe et al. (2025).

Knowns vs. unknowns

- $\boldsymbol{x}$: per-sample data (e.g. lattice mask $x_{ij} \in \{0,1\}$).
- $\boldsymbol{\alpha} \in [0,1]^d$: *unknown, fixed* global couplings (transverse field, mass, gauge coupling, ...).

Central question

Evaluating $f_{\boldsymbol{\alpha}}(\boldsymbol{x})$ is generically BQP-hard $\Rightarrow$ a classical learner cannot even generate its own labels. *Can quantum access to the dynamics yield a provably efficient PAC learner where no classical algorithm can?*

Probably Approximately Correct (PAC) Learning — definition

Originally introduced by L. Valiant (1984); we use the *efficient, distribution-independent* formulation of Barthe *et al.* (Def. 1).

Definition 1 – Efficient PAC learnability (Barthe *et al.*, 2025)

A concept class $\mathcal{C} = \{\mathcal{C}_n\}_n$ with $c : \mathcal{X}_n \to \mathcal{Y}_n \in \mathcal{C}_n$ is *efficiently PAC-learnable* iff there exists an algorithm $\mathcal{A}$ such that,

$$\forall\, n,\ \forall\, \varepsilon{>}0,\ \forall\, \delta{\in}(0,1],\ \forall\, c \in \mathcal{C}_n,\ \forall \text{ target distribution } \mathcal{D} \text{ on } \mathcal{X}_n,$$

given an i.i.d. training dataset $\{(\boldsymbol{x}_t, c(\boldsymbol{x}_t))\}_{t=1}^{T} \sim \mathcal{D}^T$ of size $T \in \mathrm{poly}(n, \varepsilon^{-1}, \delta^{-1})$, $\mathcal{A}$ returns a hypothesis $h = \mathcal{A}(T, \varepsilon, \delta)$ in time $\mathrm{poly}(n, \varepsilon^{-1}, \delta^{-1})$ satisfying

$$\Pr_{\boldsymbol{x}\sim\mathcal{D}}\Big[\, |h(\boldsymbol{x}) - c(\boldsymbol{x})| \le \varepsilon \,\Big] \ \ge\ 1 - \delta.$$

Reading guide. *Approximately* = error $\le \varepsilon$; *Probably* = with confidence $\ge 1 - \delta$; *Efficient* = sample size *and* runtime polynomial in $(n, \varepsilon^{-1}, \delta^{-1})$; *Distribution-independent* = the bound must hold for *any* $\mathcal{D}$.

$\Longrightarrow$ a much stronger requirement than typical statistical-learning average-case guarantees.

PAC Learning — quantum vs. classical, separation target

PAC learnability is parameterized by the *model of computation* granted to the learner; everything else (samples, distribution, guarantees) is identical.

Classical-efficient PAC	Quantum-efficient PAC
$\mathcal{A}$ and h are uniform classical poly-time TMs (or non-uniform: belong to P/poly).	$\mathcal{A}$ is a uniform quantum poly-time circuit; the hypothesis h may be evaluated by a quantum sub-routine on a fresh $\boldsymbol{x}$.
Lower bounds typically come from *cryptographic* or *complexity-theoretic* hardness.	Same PAC bound; the only relaxation is the hardware.

The labels are quantum either way. Both learners receive the same dataset $\{(\boldsymbol{x}_t, c(\boldsymbol{x}_t))\}_t$; generating $c(\boldsymbol{x}_t)$ for the concept classes we will consider already requires BQP power.

Target separation (Barthe *et al.*, Thm. 2 & Thm. 3)

Exhibit a family $\mathcal{C}$ that is efficiently *quantum* PAC-learnable *and*, under the standard complexity hypothesis $\boxed{\mathrm{BQP} \not\subseteq \mathrm{P/poly}}$, *not* efficiently classically PAC-learnable.

$\Rightarrow$ provable, super-polynomial quantum-classical learning advantage on a *natural, physically motivated* task.

Pauli-encoded PQCs — state-level Fourier structure

Definition 2 – Pauli encoding (Barthe *et al.*, 2025)

A *Pauli-encoded circuit* on n qubits is a unitary $U : \boldsymbol{\alpha} \in [0,1]^d \to U(2^n)$ composed of $N_f \in \mathrm{poly}(n)$ fixed gates and $L \in \mathrm{poly}(n)$ parametrized gates

$$V_l(\boldsymbol{\alpha}) \;:=\; e^{i\pi\, P_l\, \alpha_{i_l}}, \qquad P_l \in \{I, X, Y, Z\}^{\otimes n}, \;\; i_l \in \{1, \ldots, d\}.$$

The same parameter α_{i_l} may be *re-uploaded* (used in several V_l's) – essential for Trotterized evolutions.

Pauli-encoded PQCs — state-level Fourier structure

Definition 2 – Pauli encoding (Barthe *et al.*, 2025)

A *Pauli-encoded circuit* on n qubits is a unitary $U : \boldsymbol{\alpha} \in [0,1]^d \to U(2^n)$ composed of $N_f \in \mathrm{poly}(n)$ fixed gates and $L \in \mathrm{poly}(n)$ parametrized gates

$$V_l(\boldsymbol{\alpha}) \;:=\; e^{i\pi\, P_l\, \alpha_{i_l}}, \qquad P_l \in \{I, X, Y, Z\}^{\otimes n}, \;\; i_l \in \{1, \ldots, d\}.$$

The same parameter α_{i_l} may be *re-uploaded* (used in several V_l's) – essential for Trotterized evolutions.

Lemma 1 – Fourier representation of $U(\boldsymbol{\alpha})\,|0\rangle$ (Schuld–Sweke–Meyer, 2021)

Any Pauli-encoded circuit produces a state that is a *finite trigonometric polynomial* in $\boldsymbol{\alpha}$:

$$|\phi(\boldsymbol{\alpha})\rangle \;=\; U(\boldsymbol{\alpha})\,|0\rangle \;=\; \sum_{k=1}^{2^n} \sum_{\boldsymbol{l} \in [-L,+L]^d} a_{\boldsymbol{l},k}\, e^{i\pi\, \boldsymbol{\alpha}\cdot\boldsymbol{l}}\, |k\rangle\,, \qquad a_{\boldsymbol{l},k} \in \mathbb{C}.$$

Unknowns $\boldsymbol{\alpha}$ appear *only as global phases*; the amplitudes $a_{\boldsymbol{l},k}$ are $\boldsymbol{\alpha}$-independent structural data.

Pauli-encoded PQCs — state-level Fourier structure

Definition 2 – Pauli encoding (Barthe *et al.*, 2025)

A *Pauli-encoded circuit* on n qubits is a unitary $U : \boldsymbol{\alpha} \in [0,1]^d \to U(2^n)$ composed of $N_f \in \mathrm{poly}(n)$ fixed gates and $L \in \mathrm{poly}(n)$ parametrized gates

$$V_l(\boldsymbol{\alpha}) \;:=\; e^{i\pi\, P_l\, \alpha_{i_l}}, \qquad P_l \in \{I, X, Y, Z\}^{\otimes n}, \;\; i_l \in \{1, \ldots, d\}.$$

The same parameter α_{i_l} may be *re-uploaded* (used in several V_l's) – essential for Trotterized evolutions.

Lemma 1 – Fourier representation of $U(\boldsymbol{\alpha})\,|0\rangle$ (Schuld–Sweke–Meyer, 2021)

Any Pauli-encoded circuit produces a state that is a *finite trigonometric polynomial* in $\boldsymbol{\alpha}$:

$$|\phi(\boldsymbol{\alpha})\rangle \;=\; U(\boldsymbol{\alpha})\,|0\rangle \;=\; \sum_{k=1}^{2^n} \sum_{\boldsymbol{l} \in [-L,+L]^d} a_{\boldsymbol{l},k}\, e^{i\pi\, \boldsymbol{\alpha}\cdot\boldsymbol{l}}\, |k\rangle\,, \qquad a_{\boldsymbol{l},k} \in \mathbb{C}.$$

Unknowns $\boldsymbol{\alpha}$ appear *only as global phases*; the amplitudes $a_{\boldsymbol{l},k}$ are $\boldsymbol{\alpha}$-independent structural data.

$\Rightarrow$ every Trotter / qubitization / LCU realisation of $e^{iH(\boldsymbol{x},\boldsymbol{\alpha})t}$ belongs to Definition 2; the theory will transfer to physical Hamiltonian dynamics in Sec. V.

Pauli-encoded PQCs — expectation values as Fourier series

Corollary 1 – PQC function (Eqs. (4)–(8) of Barthe *et al.*)

For a Pauli observable P, the PQC function

$$f(\boldsymbol{\alpha}) \;=\; \langle 0|\, U^{\dagger}(\boldsymbol{\alpha})\, P\, U(\boldsymbol{\alpha})\, |0\rangle \;=\; \sum_{\boldsymbol{l}\in\mathcal{L}} b_{\boldsymbol{l}}\, e^{\,i\pi\,\boldsymbol{\alpha}\cdot\boldsymbol{l}}, \qquad \mathcal{L} := [-2L, +2L]^d,$$

is a finite trigonometric polynomial with at most $m := (4L+1)^d$ frequencies, coefficients $\boldsymbol{b} = (b_{\boldsymbol{l}})_{\boldsymbol{l}\in\mathcal{L}} \in \mathbb{C}^m$.

Lattice doubling $L \to 2L$. Plug Lemma 1 into $\langle\phi(\boldsymbol{\alpha})|\, P\, |\phi(\boldsymbol{\alpha})\rangle$:

Pauli-encoded PQCs — expectation values as Fourier series

Corollary 1 – PQC function (Eqs. (4)–(8) of Barthe *et al.*)

For a Pauli observable P, the PQC function

$$f(\boldsymbol{\alpha}) \;=\; \langle 0|\, U^{\dagger}(\boldsymbol{\alpha})\, P\, U(\boldsymbol{\alpha})\, |0\rangle \;=\; \sum_{\boldsymbol{l}\in\mathcal{L}} b_{\boldsymbol{l}}\, e^{\,i\pi\,\boldsymbol{\alpha}\cdot\boldsymbol{l}}, \qquad \mathcal{L} := [-2L, +2L]^d,$$

is a finite trigonometric polynomial with at most $m := (4L+1)^d$ frequencies, coefficients $\boldsymbol{b} = (b_{\boldsymbol{l}})_{\boldsymbol{l}\in\mathcal{L}} \in \mathbb{C}^m$.

Lattice doubling $L \to 2L$. Plug Lemma 1 into $\langle\phi(\boldsymbol{\alpha})|\, P\, |\phi(\boldsymbol{\alpha})\rangle$:

$$f(\boldsymbol{\alpha}) = \sum_{k,k',\boldsymbol{l},\boldsymbol{l}'} \overline{a_{\boldsymbol{l},k}}\, a_{\boldsymbol{l}',k'}\, P_{kk'}\, e^{\,i\pi\boldsymbol{\alpha}\cdot(\boldsymbol{l}'-\boldsymbol{l})}.$$

Pauli-encoded PQCs — expectation values as Fourier series

Corollary 1 – PQC function (Eqs. (4)–(8) of Barthe *et al.*)

For a Pauli observable P, the PQC function

$$f(\boldsymbol{\alpha}) \;=\; \langle 0|\, U^{\dagger}(\boldsymbol{\alpha})\, P\, U(\boldsymbol{\alpha})\, |0\rangle \;=\; \sum_{\boldsymbol{l}\in\mathcal{L}} b_{\boldsymbol{l}}\, e^{\,i\pi\,\boldsymbol{\alpha}\cdot\boldsymbol{l}}, \qquad \mathcal{L} := [-2L, +2L]^d,$$

is a finite trigonometric polynomial with at most $m := (4L+1)^d$ frequencies, coefficients $\boldsymbol{b} = (b_{\boldsymbol{l}})_{\boldsymbol{l}\in\mathcal{L}} \in \mathbb{C}^m$.

Lattice doubling $L \to 2L$. Plug Lemma 1 into $\langle\phi(\boldsymbol{\alpha})|\, P\, |\phi(\boldsymbol{\alpha})\rangle$:

$$f(\boldsymbol{\alpha}) = \sum_{k,k',\boldsymbol{l},\boldsymbol{l}'} \overline{a_{\boldsymbol{l},k}}\, a_{\boldsymbol{l}',k'}\, P_{kk'}\, e^{\,i\pi\boldsymbol{\alpha}\cdot(\boldsymbol{l}'-\boldsymbol{l})}.$$

Group terms by $\boldsymbol{l} := \boldsymbol{l}' - \boldsymbol{l}$: $\boldsymbol{l} \in [-L, L]^d - [-L, L]^d = \mathcal{L}$,
$b_{\boldsymbol{l}} = \sum_{k,k'} \sum_{\boldsymbol{l}'-\boldsymbol{l}''=\boldsymbol{l}} \overline{a_{\boldsymbol{l}'',k}}\, a_{\boldsymbol{l}',k'}\, P_{kk'}$.

Pauli-encoded PQCs — expectation values as Fourier series

Corollary 1 – PQC function (Eqs. (4)–(8) of Barthe *et al.*)

For a Pauli observable P, the PQC function

$$f(\boldsymbol{\alpha}) \;=\; \langle 0|\, U^{\dagger}(\boldsymbol{\alpha})\, P\, U(\boldsymbol{\alpha})\, |0\rangle \;=\; \sum_{\boldsymbol{l}\in\mathcal{L}} b_{\boldsymbol{l}}\, e^{i\pi\,\boldsymbol{\alpha}\cdot\boldsymbol{l}}, \qquad \mathcal{L} := [-2L, +2L]^d,$$

is a finite trigonometric polynomial with at most $m := (4L+1)^d$ frequencies, coefficients $\boldsymbol{b} = (b_{\boldsymbol{l}})_{\boldsymbol{l}\in\mathcal{L}} \in \mathbb{C}^m$.

Lattice doubling $L \to 2L$. Plug Lemma 1 into $\langle\phi(\boldsymbol{\alpha})|\, P\, |\phi(\boldsymbol{\alpha})\rangle$:

$$f(\boldsymbol{\alpha}) = \sum_{k,k',\boldsymbol{l},\boldsymbol{l}'} \overline{a_{\boldsymbol{l},k}}\, a_{\boldsymbol{l}',k'}\, P_{kk'}\, e^{i\pi\boldsymbol{\alpha}\cdot(\boldsymbol{l}'-\boldsymbol{l})}.$$

Group terms by $\boldsymbol{l} := \boldsymbol{l}' - \boldsymbol{l}$: $\boldsymbol{l} \in [-L, L]^d - [-L, L]^d = \mathcal{L}$, $b_{\boldsymbol{l}} = \sum_{k,k'} \sum_{\boldsymbol{l}'-\boldsymbol{l}''=\boldsymbol{l}} \overline{a_{\boldsymbol{l}'',k}}\, a_{\boldsymbol{l}',k'}\, P_{kk'}$.

Structural takeaways.

- Finite integer lattice $\mathcal{L} = [-2L, 2L]^d$, $|\mathcal{L}| = (4L+1)^d$.
- Unknowns appear *only* in phases $e^{i\pi\boldsymbol{\alpha}\cdot\boldsymbol{l}}$: structure and parameters decouple.
- Learning $f \Leftrightarrow$ learning $\boldsymbol{b} \in \mathbb{C}^m \Rightarrow$ linear regression once $\boldsymbol{b}$ is exposed.

The PQC concept class — Definition 3

Definition 3 – PQC-based functions concept class (Barthe *et al.*)

Let $U(\boldsymbol{x}, \boldsymbol{\alpha})$ be a Pauli-encoded circuit on n qubits in which $\boldsymbol{x} \in \{0,1\}^N$ bit-encodes the *fixed* sub-circuit (lattice mask, geometry, ...) and $\boldsymbol{\alpha} \in [0,1]^{d_n}$, $d_n = f(n)$, are the unknown shared parameters. For an observable O,

$$\mathcal{C}_{n,f}^{\mathrm{PQC}} \;=\; \left\{ \boldsymbol{x} \mapsto \langle 0| \, U^\dagger(\boldsymbol{x}, \boldsymbol{\alpha}) \, O \, U(\boldsymbol{x}, \boldsymbol{\alpha}) \, |0\rangle \;:\; \boldsymbol{\alpha} \in [0,1]^{d_n} \right\}.$$

Two regimes: $f(n) = \log n \Rightarrow \mathcal{C}_{\log}^{\mathrm{PQC}}$ (PAC-learnable); $f(n) = \mathrm{poly}(n) \Rightarrow \mathcal{C}_{\mathrm{poly}}^{\mathrm{PQC}}$ (hard).

Core identity (Eq. (8) with x tracked)

Sandwich O and group by frequency as on Slide 4b, tracking the $\boldsymbol{x}$-dependence of the amplitudes $a_{\boldsymbol{l},k}(\boldsymbol{x})$:

$$f_{\boldsymbol{\alpha}}(\boldsymbol{x}) \;=\; \sum_{\boldsymbol{l} \in \mathcal{L}} b_{\boldsymbol{l}}(\boldsymbol{x}) \, e^{i\pi \, \boldsymbol{\alpha} \cdot \boldsymbol{l}}, \qquad \mathcal{L} = [-2L, +2L]^{d_n}, \; |\mathcal{L}| = m.$$

$\boldsymbol{b}(\boldsymbol{x}) = (b_{\boldsymbol{l}}(\boldsymbol{x}))_{\boldsymbol{l} \in \mathcal{L}} \in \mathbb{C}^m$ is an $\boldsymbol{x}$-dependent feature vector; phases are $\boldsymbol{x}$-independent.

$\Rightarrow$ the PAC dataset $\{(\boldsymbol{x}_t, f_{\boldsymbol{\alpha}}(\boldsymbol{x}_t))\}$ inherits the structure of a generalized linear model in the latent features $\boldsymbol{b}(\boldsymbol{x}_t)$.

Learning reduction — features and phases decouple

The core identity is a complex *linear* model in two dual feature maps – one extracted from the data, one carrying the unknowns.

Bilinear factorization

$$f_{\boldsymbol{\alpha}}(\boldsymbol{x}) \;=\; \boldsymbol{b}(\boldsymbol{x}) \cdot \boldsymbol{w}(\boldsymbol{\alpha}), \qquad \boldsymbol{w}(\boldsymbol{\alpha}) = \left(e^{i\pi\, \boldsymbol{l}\cdot\boldsymbol{\alpha}}\right)_{\boldsymbol{l}\in\mathcal{L}} \in \mathbb{C}^m, \;\; \boldsymbol{b}(\boldsymbol{x}) \in \mathbb{C}^m.$$

$\boldsymbol{b}(\boldsymbol{x})$: *known* per-sample feature vector, *computable* by Algorithm A (Sec. III, next slides).

$\boldsymbol{w}(\boldsymbol{\alpha})$: *unknown* but $\boldsymbol{x}$-independent; lies on the m-torus $\left\{ e^{i\pi \boldsymbol{l}\cdot\boldsymbol{\alpha}} \right\}_{\boldsymbol{l}}$, with $\|\boldsymbol{w}\|_\infty = 1$, $\|\boldsymbol{w}\|_1 \le m$.

Quantum–classical pipeline.

Learning reduction — features and phases decouple

The core identity is a complex *linear* model in two dual feature maps – one extracted from the data, one carrying the unknowns.

Bilinear factorization

$$f_{\boldsymbol{\alpha}}(\boldsymbol{x}) \;=\; \boldsymbol{b}(\boldsymbol{x}) \cdot \boldsymbol{w}(\boldsymbol{\alpha}), \qquad \boldsymbol{w}(\boldsymbol{\alpha}) = \left(e^{i\pi\, \boldsymbol{l}\cdot\boldsymbol{\alpha}}\right)_{\boldsymbol{l}\in\mathcal{L}} \in \mathbb{C}^m, \;\; \boldsymbol{b}(\boldsymbol{x}) \in \mathbb{C}^m.$$

$\boldsymbol{b}(\boldsymbol{x})$: *known* per-sample feature vector, *computable* by Algorithm A (Sec. III, next slides).

$\boldsymbol{w}(\boldsymbol{\alpha})$: *unknown* but $\boldsymbol{x}$-independent; lies on the m-torus $\left\{ e^{i\pi \boldsymbol{l}\cdot\boldsymbol{\alpha}} \right\}_{\boldsymbol{l}}$, with $\|\boldsymbol{w}\|_\infty = 1$, $\|\boldsymbol{w}\|_1 \le m$.

Quantum–classical pipeline.

1. *Quantum, per sample*: run Algorithm A on $U(\boldsymbol{x}_t, \cdot)$ and Hadamard-test it to retrieve $\hat{\boldsymbol{b}}(\boldsymbol{x}_t)$ up to additive error ε_b.

Learning reduction — features and phases decouple

The core identity is a complex *linear* model in two dual feature maps – one extracted from the data, one carrying the unknowns.

Bilinear factorization

$$f_{\boldsymbol{\alpha}}(\boldsymbol{x}) \;=\; \boldsymbol{b}(\boldsymbol{x}) \cdot \boldsymbol{w}(\boldsymbol{\alpha}), \qquad \boldsymbol{w}(\boldsymbol{\alpha}) = \left(e^{i\pi\, \boldsymbol{l}\cdot\boldsymbol{\alpha}}\right)_{\boldsymbol{l}\in\mathcal{L}} \in \mathbb{C}^m, \;\; \boldsymbol{b}(\boldsymbol{x}) \in \mathbb{C}^m.$$

$\boldsymbol{b}(\boldsymbol{x})$: *known* per-sample feature vector, *computable* by Algorithm A (Sec. III, next slides).

$\boldsymbol{w}(\boldsymbol{\alpha})$: *unknown* but $\boldsymbol{x}$-independent; lies on the m-torus $\left\{ e^{i\pi \boldsymbol{l}\cdot\boldsymbol{\alpha}} \right\}_{\boldsymbol{l}}$, with $\|\boldsymbol{w}\|_\infty = 1$, $\|\boldsymbol{w}\|_1 \le m$.

Quantum–classical pipeline.

1. *Quantum, per sample*: run Algorithm A on $U(\boldsymbol{x}_t, \cdot)$ and Hadamard-test it to retrieve $\hat{\boldsymbol{b}}(\boldsymbol{x}_t)$ up to additive error ε_b.
2. *Classical, once*: solve the regression

$$\hat{\boldsymbol{w}} \;=\; \arg \min_{\|\boldsymbol{w}\|_1 \le \Lambda_1} \left\| \boldsymbol{y} - \hat{B}\, \boldsymbol{w} \right\|_2^2, \;\; \hat{B}_{t,\boldsymbol{l}} = \hat{b}_{\boldsymbol{l}}(\boldsymbol{x}_t).$$

Learning reduction — features and phases decouple

The core identity is a complex *linear* model in two dual feature maps – one extracted from the data, one carrying the unknowns.

Bilinear factorization

$$f_{\boldsymbol{\alpha}}(\boldsymbol{x}) \;=\; \boldsymbol{b}(\boldsymbol{x}) \cdot \boldsymbol{w}(\boldsymbol{\alpha}), \qquad \boldsymbol{w}(\boldsymbol{\alpha}) = \left(e^{i\pi\, \boldsymbol{l}\cdot\boldsymbol{\alpha}}\right)_{\boldsymbol{l}\in\mathcal{L}} \in \mathbb{C}^m, \;\; \boldsymbol{b}(\boldsymbol{x}) \in \mathbb{C}^m.$$

$\boldsymbol{b}(\boldsymbol{x})$: *known* per-sample feature vector, *computable* by Algorithm A (Sec. III, next slides).

$\boldsymbol{w}(\boldsymbol{\alpha})$: *unknown* but $\boldsymbol{x}$-independent; lies on the m-torus $\left\{ e^{i\pi \boldsymbol{l}\cdot\boldsymbol{\alpha}} \right\}_{\boldsymbol{l}}$, with $\|\boldsymbol{w}\|_\infty = 1$, $\|\boldsymbol{w}\|_1 \le m$.

Quantum–classical pipeline.

1. *Quantum, per sample*: run Algorithm A on $U(\boldsymbol{x}_t, \cdot)$ and Hadamard-test it to retrieve $\hat{\boldsymbol{b}}(\boldsymbol{x}_t)$ up to additive error ε_b.
2. *Classical, once*: solve the regression

$$\hat{\boldsymbol{w}} \;=\; \arg \min_{\|\boldsymbol{w}\|_1 \le \Lambda_1} \left\| \boldsymbol{y} - \hat{B}\, \boldsymbol{w} \right\|_2^2, \;\; \hat{B}_{t,\boldsymbol{l}} = \hat{b}_{\boldsymbol{l}}(\boldsymbol{x}_t).$$

3. *Inference*: $h(\boldsymbol{x}) = \hat{\boldsymbol{b}}(\boldsymbol{x}) \cdot \hat{\boldsymbol{w}}$.

Learning reduction — features and phases decouple

The core identity is a complex *linear* model in two dual feature maps – one extracted from the data, one carrying the unknowns.

Bilinear factorization

$$f_{\boldsymbol{\alpha}}(\boldsymbol{x}) \;=\; \boldsymbol{b}(\boldsymbol{x}) \cdot \boldsymbol{w}(\boldsymbol{\alpha}), \qquad \boldsymbol{w}(\boldsymbol{\alpha}) = \left(e^{i\pi\, \boldsymbol{l}\cdot\boldsymbol{\alpha}}\right)_{\boldsymbol{l}\in\mathcal{L}} \in \mathbb{C}^m, \;\; \boldsymbol{b}(\boldsymbol{x}) \in \mathbb{C}^m.$$

$\boldsymbol{b}(\boldsymbol{x})$: *known* per-sample feature vector, *computable* by Algorithm A (Sec. III, next slides).

$\boldsymbol{w}(\boldsymbol{\alpha})$: *unknown* but $\boldsymbol{x}$-independent; lies on the m-torus $\left\{ e^{i\pi \boldsymbol{l}\cdot\boldsymbol{\alpha}} \right\}_{\boldsymbol{l}}$, with $\|\boldsymbol{w}\|_\infty = 1$, $\|\boldsymbol{w}\|_1 \le m$.

Quantum–classical pipeline.

1. *Quantum, per sample*: run Algorithm A on $U(\boldsymbol{x}_t, \cdot)$ and Hadamard-test it to retrieve $\hat{\boldsymbol{b}}(\boldsymbol{x}_t)$ up to additive error ε_b.
2. *Classical, once*: solve the regression

$$\hat{\boldsymbol{w}} \;=\; \arg\min_{\|\boldsymbol{w}\|_1 \le \Lambda_1} \left\| \boldsymbol{y} - \hat{B}\, \boldsymbol{w} \right\|_2^2, \;\; \hat{B}_{t,\boldsymbol{l}} = \hat{b}_{\boldsymbol{l}}(\boldsymbol{x}_t).$$

3. *Inference*: $h(\boldsymbol{x}) = \hat{\boldsymbol{b}}(\boldsymbol{x}) \cdot \hat{\boldsymbol{w}}$.

Why this reduction is the whole game.

- Eliminates $\boldsymbol{\alpha}$ from the model: the regressor never touches the unknown couplings.
- Sample complexity controlled by $\|\boldsymbol{w}\|_1$ (LASSO, Theorem 6), *not* by $|\mathcal{L}|$.
- Hard part is now purely quantum: extract $\boldsymbol{b}(\boldsymbol{x})$ – the topic of Algorithm A.

Algorithm A — architecture and register design

Objective (Theorem 1, Sec. III of Barthe *et al.*)

Compile any Pauli-encoded $U(\boldsymbol{x}, \boldsymbol{\alpha})$ into a *parameter-free* circuit $A(U)$ on a larger Hilbert space such that

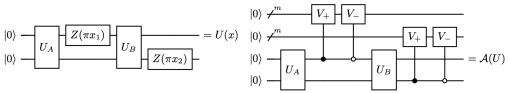
$$A(U)\ |0\rangle_f\, |0\rangle_a\, |0\rangle_c \;=\; \sum_{k=1}^{2^n} \sum_{\boldsymbol{l}\in[-L,L]^d} a_{\boldsymbol{l},k}(\boldsymbol{x})\ |\boldsymbol{l}\rangle_f\ |0\rangle_a\ |k\rangle_c\,.$$

Each Fourier index $\boldsymbol{l}$ is now a *physical, addressable* basis state – we can post-select or Hadamard-test on it.

Three registers $\left[\, n_T = n + 1 + \sum_s n_s \,\right]$

- c circuit register, n qubits.
- a parity ancilla, 1 qubit.
- $f = f_1 \cdots f_d$ frequency register; $n_s = 1 + \lceil 2 \log s \rceil$ qubits per dimension, with circular-boundary increment / decrement gates $V_+, V_- = V_+^\dagger$.

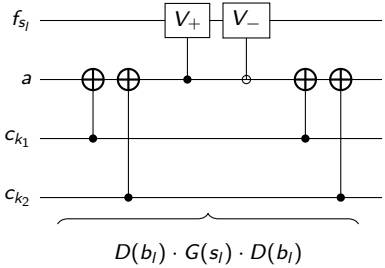
Resource counts (Theorem 1). Total
$n' = n + d\lceil \log L \rceil + 1$ qubits,
$L' = N_f + L(2n + d\lceil \log L \rceil)$ gates; *all* α-independent.



Left: a 2-qubit Pauli-encoded $U(\boldsymbol{x})$. Right: the compiled $A(U)$. The $V_\pm$ blocks per encoding gate stand in for the D·G·D construction (next slide). Figure 1 of Barthe et al., 2025.

Algorithm A — the $D \cdot G \cdot D$ transformation

Setup. WLOG every encoding gate is $V_l(\boldsymbol{\alpha}) = \exp\left(i\pi\alpha_{s_l} P_l\right)$ with $P_l = \prod_{k:b_{l,k}=1} Z_k$ (X, Y are absorbed into the fixed gates). Algorithm A replaces it by $\boxed{V_l(\boldsymbol{\alpha}) \longmapsto D(b_l) \cdot G(s_l) \cdot D(b_l)}$

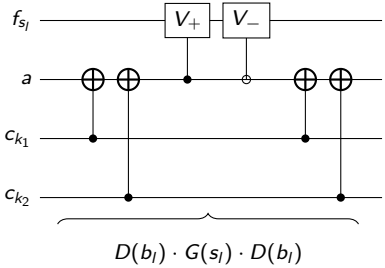


One block; two relevant c-qubits ($b_{l,k_1} = b_{l,k_2} = 1$). The two D cascades sandwich G and reset a.

Algorithm A — the $D \cdot G \cdot D$ transformation

Setup. WLOG every encoding gate is $V_l(\boldsymbol{\alpha}) = \exp\!\left(i\pi\alpha_{s_l} P_l\right)$ with $P_l = \prod_{k:b_{l,k}=1} Z_k$ (X, Y are absorbed into the fixed gates). Algorithm A replaces it by $\boxed{V_l(\boldsymbol{\alpha}) \longmapsto D(b_l) \cdot G(s_l) \cdot D(b_l)}$

$D(b_l)$ CNOT cascade $\prod_{k:b_{l,k}=1} \mathrm{CNOT}^{(c_k \to a)}$, writes parity $p_l(k) = \bigoplus_{j:b_{l,j}=1} k_j \in \{0,1\}$ onto a.



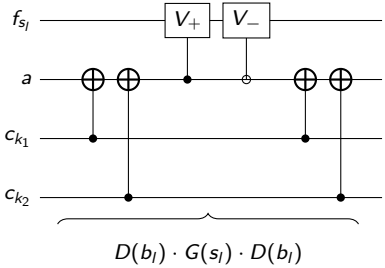
One block; two relevant c-qubits ($b_{l,k_1} = b_{l,k_2} = 1$). The two D cascades sandwich G and reset a.

Algorithm A — the $D \cdot G \cdot D$ transformation

Setup. WLOG every encoding gate is $V_l(\boldsymbol{\alpha}) = \exp\!\left(i\pi\alpha_{s_l} P_l\right)$ with $P_l = \prod_{k:b_{l,k}=1} Z_k$ (X, Y are absorbed into the fixed gates). Algorithm A replaces it by $\boxed{V_l(\boldsymbol{\alpha}) \longmapsto D(b_l) \cdot G(s_l) \cdot D(b_l)}$

$D(b_l)$ CNOT cascade $\prod_{k:b_{l,k}=1} \mathrm{CNOT}^{(c_k \to a)}$, writes parity $p_l(k) = \bigoplus_{j:b_{l,j}=1} k_j \in \{0,1\}$ onto a.

$G(s_l)$ parity-controlled increment / decrement of f_{s_l}: apply V_+ when $p_l = 1$, V_- when $p_l = 0$.



One block; two relevant c-qubits ($b_{l,k_1} = b_{l,k_2} = 1$). The two D cascades sandwich G and reset a.

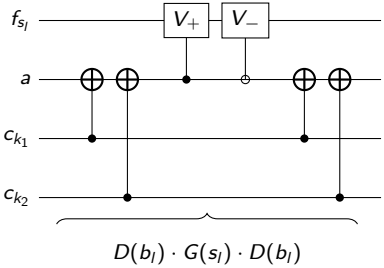
Algorithm A — the $D \cdot G \cdot D$ transformation

Setup. WLOG every encoding gate is $V_l(\boldsymbol{\alpha}) = \exp\!\left(i\pi\alpha_{s_l} P_l\right)$ with $P_l = \prod_{k:b_{l,k}=1} Z_k$ (X, Y are absorbed into the fixed gates). Algorithm A replaces it by $\boxed{V_l(\boldsymbol{\alpha}) \longmapsto D(b_l) \cdot G(s_l) \cdot D(b_l)}$

$D(b_l)$ CNOT cascade $\prod_{k:b_{l,k}=1} \mathrm{CNOT}^{(c_k \to a)}$, writes parity $p_l(k) = \bigoplus_{j:b_{l,j}=1} k_j \in \{0,1\}$ onto a.

$G(s_l)$ parity-controlled increment / decrement of f_{s_l}: apply V_+ when $p_l = 1$, V_- when $p_l = 0$.

$D(b_l)$ second cascade uncomputes a to $|0\rangle$ ($D^2 = I$), so the ancilla is reusable.



One block; two relevant c-qubits ($b_{l,k_1} = b_{l,k_2} = 1$). The two D cascades sandwich G and reset a.

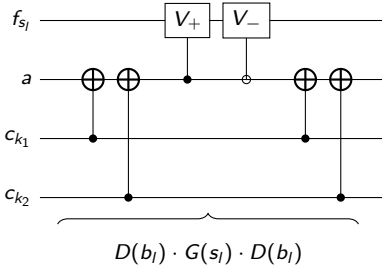
Algorithm A — the $D \cdot G \cdot D$ transformation

Setup. WLOG every encoding gate is $V_l(\boldsymbol{\alpha}) = \exp\big(i\pi\alpha_{s_l} P_l\big)$ with $P_l = \prod_{k:b_{l,k}=1} Z_k$ (X, Y are absorbed into the fixed gates). Algorithm A replaces it by $\boxed{V_l(\boldsymbol{\alpha}) \longmapsto D(b_l) \cdot G(s_l) \cdot D(b_l)}$

$D(b_l)$ CNOT cascade $\prod_{k:b_{l,k}=1} \mathrm{CNOT}^{(c_k \to a)}$, writes parity $p_l(k) = \bigoplus_{j:b_{l,j}=1} k_j \in \{0,1\}$ onto a.

$G(s_l)$ parity-controlled increment / decrement of f_{s_l}: apply V_+ when $p_l = 1$, V_- when $p_l = 0$.

$D(b_l)$ second cascade uncomputes a to $|0\rangle$ ($D^2 = I$), so the ancilla is reusable.



One block; two relevant c-qubits ($b_{l,k_1} = b_{l,k_2} = 1$). The two D cascades sandwich G and reset a.

Net effect. The phase $e^{i\pi\alpha_{s_l}\sigma_l(k)}$ of $V_l(\boldsymbol{\alpha})$ becomes an additive shift of f_{s_l}:

$$|l\rangle_f\,|0\rangle_a\,|k\rangle_c \longmapsto |l + e_{s_l}\sigma_l(k)\rangle_f\,|0\rangle_a\,|k\rangle_c\,,\quad \sigma_l = (-1)^{1 \oplus p_l}.$$

Accumulated over the L gates this builds $\boldsymbol{I}$ of Lemma 1 (correctness proof next slide).

Algorithm A — correctness by induction (Theorem 1)

Theorem 1 (Barthe *et al.*, Sec. III)

For every Pauli-encoded U, the parameter-free circuit $A(U)$ prepares $|\phi'\rangle = A(U)\,|0\rangle = \sum_{k,\boldsymbol{l}\in[-L,L]^d} a_{\boldsymbol{l},k}\ |\boldsymbol{l}\rangle_f\,|0\rangle_a\,|k\rangle_c$ in $\mathrm{poly}(n,d)$ qubits/gates, with *no knowledge of* $\boldsymbol{\alpha}$. Equivalently, A physically tags the Fourier indices of Lemma 1.

Algorithm A — correctness by induction (Theorem 1)

Theorem 1 (Barthe *et al.*, Sec. III)

For every Pauli-encoded U, the parameter-free circuit $A(U)$ prepares $|\phi'\rangle = A(U)\,|0\rangle = \sum_{k,\boldsymbol{l}\in[-L,L]^d} a_{\boldsymbol{l},k}\,|\boldsymbol{l}\rangle_f\,|0\rangle_a\,|k\rangle_c$ in $\mathrm{poly}(n,d)$ qubits/gates, with *no knowledge of* $\boldsymbol{\alpha}$. Equivalently, A physically tags the Fourier indices of Lemma 1.

Proof. Induction on the number of processed gates, with invariant $|\phi'_t\rangle = \sum_{k,\boldsymbol{l}} a^{(t)}_{\boldsymbol{l},k}\,|\boldsymbol{l}\rangle_f\,|0\rangle_a\,|k\rangle_c$, where $a^{(t)}_{\boldsymbol{l},k}$ are the Lemma 1 amplitudes of the truncated circuit $U_{\leq t}$.

Algorithm A — correctness by induction (Theorem 1)

Theorem 1 (Barthe *et al.*, Sec. III)

For every Pauli-encoded U, the parameter-free circuit $A(U)$ prepares $|\phi'\rangle = A(U)|0\rangle = \sum_{k,\boldsymbol{l}\in[-L,L]^d} a_{\boldsymbol{l},k}\, |\boldsymbol{l}\rangle_f\, |0\rangle_a\, |k\rangle_c$ in $\mathrm{poly}(n,d)$ qubits/gates, with *no knowledge of* $\boldsymbol{\alpha}$. Equivalently, A physically tags the Fourier indices of Lemma 1.

Proof. Induction on the number of processed gates, with invariant $|\phi'_t\rangle = \sum_{k,\boldsymbol{l}} a^{(t)}_{\boldsymbol{l},k}\, |\boldsymbol{l}\rangle_f\, |0\rangle_a\, |k\rangle_c$, where $a^{(t)}_{\boldsymbol{l},k}$ are the Lemma 1 amplitudes of the truncated circuit $U_{\leq t}$.

1. Base. $A(I)|0\rangle = |0\rangle_f\, |0\rangle_a\, |0\rangle_c =$
Lemma 1 for $U = I$. ✓

Algorithm A — correctness by induction (Theorem 1)

Theorem 1 (Barthe *et al.*, Sec. III)

For every Pauli-encoded U, the parameter-free circuit $A(U)$ prepares $|\phi'\rangle = A(U)\,|0\rangle = \sum_{k,\boldsymbol{l}\in[-L,L]^d} a_{\boldsymbol{l},k}\,|\boldsymbol{l}\rangle_f\,|0\rangle_a\,|k\rangle_c$ in $\mathrm{poly}(n,d)$ qubits/gates, with *no knowledge of* $\boldsymbol{\alpha}$. Equivalently, A physically tags the Fourier indices of Lemma 1.

Proof. Induction on the number of processed gates, with invariant $|\phi'_t\rangle = \sum_{k,\boldsymbol{l}} a^{(t)}_{\boldsymbol{l},k}\,|\boldsymbol{l}\rangle_f\,|0\rangle_a\,|k\rangle_c$, where $a^{(t)}_{\boldsymbol{l},k}$ are the Lemma 1 amplitudes of the truncated circuit $U_{\leq t}$.

1. Base. $A(I)\,|0\rangle = |0\rangle_f\,|0\rangle_a\,|0\rangle_c =$ Lemma 1 for $U = I$. ✓

2. Fixed gate $U^{(c)}$**.** Lifted as $I^{(f)} \otimes I^{(a)} \otimes U^{(c)}$: $a^{(t+1)}_{\boldsymbol{l},k} = \sum_{k'} U_{kk'}\,a^{(t)}_{\boldsymbol{l},k'}$, frequencies untouched. ✓

Algorithm A — correctness by induction (Theorem 1)

Theorem 1 (Barthe *et al.*, Sec. III)

For every Pauli-encoded U, the parameter-free circuit $A(U)$ prepares $|\phi'\rangle = A(U)\,|0\rangle = \sum_{k,\boldsymbol{l}\in[-L,L]^d} a_{\boldsymbol{l},k}\,|\boldsymbol{l}\rangle_f\,|0\rangle_a\,|k\rangle_c$ in $\mathrm{poly}(n,d)$ qubits/gates, with *no knowledge of* $\boldsymbol{\alpha}$. Equivalently, A physically tags the Fourier indices of Lemma 1.

Proof. Induction on the number of processed gates, with invariant $|\phi'_t\rangle = \sum_{k,\boldsymbol{l}} a^{(t)}_{\boldsymbol{l},k}\,|\boldsymbol{l}\rangle_f\,|0\rangle_a\,|k\rangle_c$, where $a^{(t)}_{\boldsymbol{l},k}$ are the Lemma 1 amplitudes of the truncated circuit $U_{\leq t}$.

1. Base. $A(I)\,|0\rangle = |0\rangle_f\,|0\rangle_a\,|0\rangle_c =$ Lemma 1 for $U = I$. ✓

2. Fixed gate $U^{(c)}$**.** Lifted as $I^{(f)} \otimes I^{(a)} \otimes U^{(c)}$: $a^{(t+1)}_{\boldsymbol{l},k} = \sum_{k'} U_{kk'}\, a^{(t)}_{\boldsymbol{l},k'}$, frequencies untouched. ✓

3. Encoding gate $\{s_l, b_l\}$**.** $D \cdot G \cdot D$ sends $a^{(t)}_{\boldsymbol{l},k} \to a^{(t+1)}_{\boldsymbol{l}+e_{s_l}\sigma_l(k),\,k}$, matching $e^{i\pi\alpha_{s_l}P_l}$ on $\sum a^{(t)} e^{i\pi\boldsymbol{\alpha}\cdot\boldsymbol{l}}\,|k\rangle$. ✓

Algorithm A — correctness by induction (Theorem 1)

Theorem 1 (Barthe *et al.*, Sec. III)

For every Pauli-encoded U, the parameter-free circuit $A(U)$ prepares $|\phi'\rangle = A(U)\,|0\rangle = \sum_{k,\boldsymbol{l}\in[-L,L]^d} a_{\boldsymbol{l},k}\,|\boldsymbol{l}\rangle_f\,|0\rangle_a\,|k\rangle_c$ in $\mathrm{poly}(n,d)$ qubits/gates, with *no knowledge of* $\boldsymbol{\alpha}$. Equivalently, A physically tags the Fourier indices of Lemma 1.

Proof. Induction on the number of processed gates, with invariant $|\phi'_t\rangle = \sum_{k,\boldsymbol{l}} a^{(t)}_{\boldsymbol{l},k}\,|\boldsymbol{l}\rangle_f\,|0\rangle_a\,|k\rangle_c$, where $a^{(t)}_{\boldsymbol{l},k}$ are the Lemma 1 amplitudes of the truncated circuit $U_{\leq t}$.

1. Base. $A(I)\,|0\rangle = |0\rangle_f\,|0\rangle_a\,|0\rangle_c =$ Lemma 1 for $U = I$. ✓

2. Fixed gate $U^{(c)}$**.** Lifted as $I^{(f)} \otimes I^{(a)} \otimes U^{(c)}$: $a^{(t+1)}_{\boldsymbol{l},k} = \sum_{k'} U_{kk'}\,a^{(t)}_{\boldsymbol{l},k'}$, frequencies untouched. ✓

3. Encoding gate $\{s_l, b_l\}$**.** $D \cdot G \cdot D$ sends $a^{(t)}_{\boldsymbol{l},k} \to a^{(t+1)}_{\boldsymbol{l}+e_{s_l}\sigma_l(k),\,k}$, matching $e^{i\pi\alpha_{s_l}P_l}$ on $\sum a^{(t)} e^{i\pi\boldsymbol{\alpha}\cdot\boldsymbol{l}}\,|k\rangle$. ✓

Why this matters

A is *deterministic* and *universal* (across Pauli-encoded U) and amplitude-encodes Lemma 1's spectrum with polynomial overhead. Sandwiching A into a Hadamard test on $|\boldsymbol{l}\rangle_f$ yields each $b_{\boldsymbol{l}}(\boldsymbol{x})$ up to additive error (Corollaries 1 & 2, next slide).

Amplitude-Encoding the Coefficients (Corollary 1)

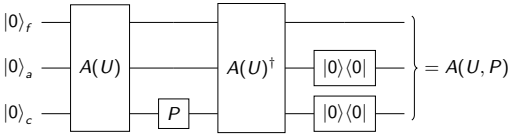
Theorem 1 lifts a circuit to its *state-level* Fourier expansion. Sandwiching $A(U)$ around an observable P lifts that further to the *expectation-value* Fourier expansion – an amplitude encoding of $\boldsymbol{b}(\boldsymbol{x})$ on the frequency register.

Corollary 1 – Expectation-value Fourier state (Eq. (9))

With $A(U,P) := \left(\langle 0|_a \langle 0|_c\right) A(U)^\dagger \, P \, A(U)$,

$$A(U,P)\ |0\rangle_f\, |0\rangle_a\, |0\rangle_c \;=\; \frac{1}{\|\boldsymbol{b}(\boldsymbol{x})\|_2} \sum_{\boldsymbol{l}\in\mathcal{L}} b_{\boldsymbol{l}}(\boldsymbol{x})\ |\boldsymbol{l}\rangle_f \;+\; |\ldots\rangle_f\, |0_\perp\rangle_{ac}\,.$$

Post-selection on $|0\rangle_a |0\rangle_c$ succeeds with probability $p_\star = \|\boldsymbol{b}(\boldsymbol{x})\|_2^2$; the surviving f-register amplitude-encodes the Fourier coefficients of $f_{\boldsymbol{\alpha}}(\boldsymbol{x}) = \langle 0|\, U^\dagger(\boldsymbol{x},\boldsymbol{\alpha})\, P\, U(\boldsymbol{x},\boldsymbol{\alpha})\, |0\rangle$.



Compound circuit $A(U,P)$; projecting $|0\rangle\langle 0|$ on a, c leaves $\boldsymbol{b}(\boldsymbol{x})$ amplitude-encoded on f. Redrawn from Fig. 2, Barthe et al. (2025).

Hadamard-Test Coefficient Estimation (Corollary 2)

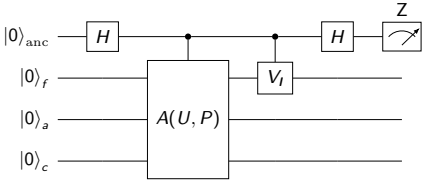
Corollary 1 amplitude-encodes $\boldsymbol{b}(\boldsymbol{x})$ on the f-register; Corollary 2 turns that into a single-number estimate of any chosen $b_{\boldsymbol{l}}$ via a controlled-$A(U,P)$ Hadamard test.

Corollary 2 – Hadamard-test extraction (Sec. III)

For every $\boldsymbol{l} \in \mathcal{L}$ there exist two $\mathrm{poly}(n,d)$-depth circuits returning $\Re b_{\boldsymbol{l}}(\boldsymbol{x})$ and $\Im b_{\boldsymbol{l}}(\boldsymbol{x})$ as the bias of a binary measurement on the ancilla; after $N = O\left(\varepsilon_b^{-2} \log \delta^{-1}\right)$ shots,

$$\Pr\Big[\ |\hat{b}_{\boldsymbol{l}}(\boldsymbol{x}) - b_{\boldsymbol{l}}(\boldsymbol{x})| \le \varepsilon_b\ \Big]\ \ge\ 1-\delta.$$

Total cost over all $|\mathcal{L}| = m$ frequencies: $O(m\,\varepsilon_b^{-2} \log \delta^{-1})$ circuit executions.



Hadamard test for $\Re b_{\boldsymbol{l}}$: the final $H \cdot Z$ gives $\Pr[0] - \Pr[1] = \Re b_{\boldsymbol{l}}$ (insert $S^\dagger$ before the last H for $\Im b_{\boldsymbol{l}}$). Redrawn from Fig. 3, Barthe et al. (2025).

Classical Stage: LASSO Regression on $\hat{b}(x)$

Algorithm A hands the regressor a noisy feature matrix $\hat{B}$ and noisy labels $\hat{\boldsymbol{y}}$; learning collapses to a constrained linear regression in the (parameter-free) features $\boldsymbol{b}(\boldsymbol{x})$.

Noise model (Eqs. A1–A3 of Barthe *et al.*)

$$\underbrace{b_l(\boldsymbol{x}_t)\cdot w_l(\boldsymbol{\alpha}) = y_t,}_{\text{linear truth}} \quad \underbrace{\hat{\boldsymbol{b}}(\boldsymbol{x}_t) = \boldsymbol{b}(\boldsymbol{x}_t) + \boldsymbol{\eta}_{b,t},\ \|\boldsymbol{\eta}_{b,t}\|_\infty \le \varepsilon_b,}_{\text{Hadamard-test noise (Cor. 2)}} \quad \underbrace{\hat{y}_t = y_t + \eta_{y,t},\ |\eta_{y,t}| \le \varepsilon_y}_{\text{label noise}}.$$

Classical Stage: LASSO Regression on $\hat{b}(x)$

Algorithm A hands the regressor a noisy feature matrix $\hat{B}$ and noisy labels $\hat{\boldsymbol{y}}$; learning collapses to a constrained linear regression in the (parameter-free) features $\boldsymbol{b}(\boldsymbol{x})$.

Noise model (Eqs. A1–A3 of Barthe *et al.*)

$$\underbrace{b_{\boldsymbol{l}}(\boldsymbol{x}_t)\cdot w_{\boldsymbol{l}}(\boldsymbol{\alpha}) = y_t,}_{\text{linear truth}} \quad \underbrace{\hat{\boldsymbol{b}}(\boldsymbol{x}_t) = \boldsymbol{b}(\boldsymbol{x}_t) + \boldsymbol{\eta}_{b,t},\ \|\boldsymbol{\eta}_{b,t}\|_\infty \le \varepsilon_b,}_{\text{Hadamard-test noise (Cor. 2)}} \quad \underbrace{\hat{y}_t = y_t + \eta_{y,t},\ |\eta_{y,t}| \le \varepsilon_y}_{\text{label noise}}.$$

LASSO objective $\left[\mathcal{H} = \{\boldsymbol{b} \mapsto \boldsymbol{b}\cdot\boldsymbol{w} \,:\, \|\boldsymbol{w}\|_1 \le \Lambda_1\}\right]$

$$\hat{\boldsymbol{w}} \;=\; \arg\min_{\|\boldsymbol{w}\|_1 \le \Lambda_1} \left\| \hat{\boldsymbol{y}} - \hat{B}\,\boldsymbol{w} \right\|_2^2, \qquad \hat{B}_{t,\boldsymbol{l}} = \hat{b}_{\boldsymbol{l}}(\boldsymbol{x}_t), \quad \Lambda_1 \ge \|\boldsymbol{w}(\boldsymbol{\alpha})\|_1.$$

Inductive bias. $\boldsymbol{w}(\boldsymbol{\alpha})$ lies on the torus $\{e^{i\pi \boldsymbol{l}\cdot\boldsymbol{\alpha}}\}$ so $\|\boldsymbol{w}\|_\infty = 1$ and Λ_1 need only scale with $m = |\mathcal{L}|$, not 2^m.

Classical Stage: LASSO Regression on $\hat{b}(x)$

Algorithm A hands the regressor a noisy feature matrix $\hat{B}$ and noisy labels $\hat{\boldsymbol{y}}$; learning collapses to a constrained linear regression in the (parameter-free) features $\boldsymbol{b}(\boldsymbol{x})$.

Noise model (Eqs. A1–A3 of Barthe *et al.*)

$$\underbrace{b_{\boldsymbol{l}}(\boldsymbol{x}_t)\cdot w_{\boldsymbol{l}}(\alpha) = y_t,}_{\text{linear truth}} \quad \underbrace{\hat{\boldsymbol{b}}(\boldsymbol{x}_t) = \boldsymbol{b}(\boldsymbol{x}_t) + \boldsymbol{\eta}_{b,t},\ \|\boldsymbol{\eta}_{b,t}\|_\infty \le \varepsilon_b,}_{\text{Hadamard-test noise (Cor. 2)}} \quad \underbrace{\hat{y}_t = y_t + \eta_{y,t},\ |\eta_{y,t}| \le \varepsilon_y}_{\text{label noise}}.$$

LASSO objective $\left[\mathcal{H} = \{\boldsymbol{b} \mapsto \boldsymbol{b}\cdot\boldsymbol{w} \,:\, \|\boldsymbol{w}\|_1 \le \Lambda_1\}\right]$

$$\hat{\boldsymbol{w}} \;=\; \arg\min_{\|\boldsymbol{w}\|_1 \le \Lambda_1} \left\| \hat{\boldsymbol{y}} - \hat{B}\,\boldsymbol{w} \right\|_2^2, \qquad \hat{B}_{t,\boldsymbol{l}} = \hat{b}_{\boldsymbol{l}}(\boldsymbol{x}_t), \quad \Lambda_1 \ge \|\boldsymbol{w}(\alpha)\|_1.$$

Inductive bias. $\boldsymbol{w}(\alpha)$ lies on the torus $\{e^{i\pi \boldsymbol{l}\cdot\alpha}\}$ so $\|\boldsymbol{w}\|_\infty = 1$ and Λ_1 need only scale with $m = |\mathcal{L}|$, not 2^m.

$\Rightarrow$ the *generalisation guarantee* that $\hat{\boldsymbol{w}}$ is close to $\boldsymbol{w}(\alpha)$ on $\mathcal{D}$-distributed test points is Theorem 6 – next slide.

Generalisation Guarantee (Theorem 6)

Theorem 6 controls the *true* mean-square risk $L_c(h) = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}[(h(\boldsymbol{x}) - c_{\boldsymbol{\alpha}}(\boldsymbol{x}))^2]$ of the LASSO minimiser $\hat{h}(\boldsymbol{x}) = \hat{\boldsymbol{w}}\cdot\hat{\boldsymbol{b}}(\boldsymbol{x})$ in terms of (i) the sample size T, (ii) the feature noise ε_b, and (iii) the label noise ε_y.

Theorem 6 – LASSO under ℓ_∞-bounded perturbations

Let $r_\infty := \sup_{\boldsymbol{x}} \|\boldsymbol{b}(\boldsymbol{x})\|_\infty$. If

$$T \;\geq\; \frac{(2\Lambda_1\, r_\infty)^4 \sqrt{2\log(2m/\delta)}}{\varepsilon^2}, \qquad \Lambda_1\,\varepsilon_b \;\leq\; 0.2\,\varepsilon, \qquad \varepsilon_y \;\leq\; 0.5\,\varepsilon,$$

then with probability at least $1-\delta$, $L_c(\hat{h}) \leq \varepsilon$.

Generalisation Guarantee (Theorem 6)

Theorem 6 controls the *true* mean-square risk $L_c(h) = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}[(h(\boldsymbol{x}) - c_\alpha(\boldsymbol{x}))^2]$ of the LASSO minimiser $\hat{h}(\boldsymbol{x}) = \hat{\boldsymbol{w}}\cdot\hat{\boldsymbol{b}}(\boldsymbol{x})$ in terms of (i) the sample size T, (ii) the feature noise ε_b, and (iii) the label noise ε_y.

Theorem 6 – LASSO under ℓ_∞-bounded perturbations

Let $r_\infty := \sup_{\boldsymbol{x}} \|\boldsymbol{b}(\boldsymbol{x})\|_\infty$. If

$$T \;\geq\; \frac{(2\Lambda_1\, r_\infty)^4 \sqrt{2\log(2m/\delta)}}{\varepsilon^2}, \qquad \Lambda_1\,\varepsilon_b \;\leq\; 0.2\,\varepsilon, \qquad \varepsilon_y \;\leq\; 0.5\,\varepsilon,$$

then with probability at least $1-\delta$, $L_c(\hat{h}) \leq \varepsilon$.

Resource trade-off.

- Sample budget: $T = \tilde{\Theta}\left((\Lambda_1 r_\infty)^4/\varepsilon^2\right)$ – *poly* in the feature dimension m only via Λ_1.
- Quantum budget: $\varepsilon_b \leq 0.2\varepsilon/\Lambda_1 \Rightarrow$ Hadamard-test $N = O(\varepsilon_b^{-2})$ shots per coefficient.
- Label budget: $\varepsilon_y \leq 0.5\varepsilon$ – absorbs Trotter / shot noise on the labels.

Generalisation Guarantee (Theorem 6)

Theorem 6 controls the *true* mean-square risk $L_c(h) = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}[(h(\boldsymbol{x}) - c_{\boldsymbol{\alpha}}(\boldsymbol{x}))^2]$ of the LASSO minimiser $\hat{h}(\boldsymbol{x}) = \hat{\boldsymbol{w}}\cdot\hat{\boldsymbol{b}}(\boldsymbol{x})$ in terms of (i) the sample size T, (ii) the feature noise ε_b, and (iii) the label noise ε_y.

Theorem 6 – LASSO under ℓ_∞-bounded perturbations

Let $r_\infty := \sup_{\boldsymbol{x}} \|\boldsymbol{b}(\boldsymbol{x})\|_\infty$. If

$$T \;\geq\; \frac{(2\Lambda_1\, r_\infty)^4 \sqrt{2\log(2m/\delta)}}{\varepsilon^2}, \qquad \Lambda_1\,\varepsilon_b \;\leq\; 0.2\,\varepsilon, \qquad \varepsilon_y \;\leq\; 0.5\,\varepsilon,$$

then with probability at least $1-\delta$, $L_c(\hat{h}) \leq \varepsilon$.

Resource trade-off.

- Sample budget: $T = \tilde{\Theta}\left((\Lambda_1 r_\infty)^4/\varepsilon^2\right)$ – *poly* in the feature dimension m only via Λ_1.
- Quantum budget: $\varepsilon_b \leq 0.2\varepsilon/\Lambda_1 \Rightarrow$ Hadamard-test $N = O(\varepsilon_b^{-2})$ shots per coefficient.
- Label budget: $\varepsilon_y \leq 0.5\varepsilon$ – absorbs Trotter / shot noise on the labels.

Specialisation to $\mathcal{C}_{\log}^{\mathbf{PQC}}$.
For the homogeneous PQC class with L re-uploads, $\Lambda_1 = m$, $r_\infty = 1$, and Theorem 6 closes to

$$T = \tilde{\Theta}\left(m^4/\varepsilon^2\right), \quad \varepsilon_b = 0.2\varepsilon/m$$

– the closed form Eq. (14) used in the proof of Theorem 2.